Aplicatie pentru cumpărarea biletelor la cursele feroviare

Student:

Cadru didactic:

Grupa II

Disciplina: Proiectarea bazelor de date

An III

Cuprins

1. Scop: domeniu, problema concreta, utilitate practica 2
Domeniu 2
Problema concretă 2
Utilitate practică 3
2. Obiective: funcționalități, operații, utilizatori 3
Utilizatori 3
Funcționalități 3
Operații 4
3. Proiectare 5
Analiza cerințelor 5
Utilizatori 5
Cerințe funcționale 5
Cerințe nonfuncționale 6
Reguli de gestiune 6
Prioritizarea cerințelor: Metoda MoSCoW 7
Descrierea entităților 8
Descrierea relațiilor 10
Modelul Logic 12
Dicționarul de date 12
Normalizarea bazei de date 16

1. Scop: domeniu, problema concreta, utilitate practica

Domeniu

Proiectul meu se încadrează în domeniul transportului feroviar de călători, mai exact în vânzarea biletelor pentru cursele de tren. Într-un astfel de sistem sunt multe informații care depind una de alta: stațiile, trenurile de tipuri diferite (InterCity, InterRegio, Regio), rutele cu opririle lor, vagoanele și locurile din ele, cursele de pe fiecare zi și biletele cumpărate de clienți.

Problema concretă

Când cumperi un bilet de tren, trebuie să afli ce trenuri circulă între două stații, la ce oră și cu ce opriri, cât costă călătoria în funcție de distanță și de tipul trenului și câte locuri mai sunt libere pe cursa aleasă. Partea mai grea este că un tren trece prin mai multe stații, iar un loc nu este ocupat pe tot traseul, ci doar de la stația unde urcă călătorul până la cea unde coboară. Același loc poate fi vândut la mai multe persoane, dacă segmentele lor de călătorie nu se suprapun. Rutele diferite în stații diferite, iar prețul depinde de distanță, de tipul trenului și de clasa vagonului.

Dacă datele nu sunt organizate bine, același loc poate fi vândut de două ori, orarele și prețurile pot ieși diferite în locuri diferite și se poate pierde istoricul biletelor. De aceea informațiile trebuie păstrate corect, iar un loc eliberat după anularea unui bilet trebuie să devină din nou disponibil.

Utilitate practică

Baza de date permite verificarea rapidă a rutelor disponibile, consultarea prețurilor, rezervarea biletelor și evidența locurilor libere. Clientul poate găsi ușor cursa potrivită, poate vedea ce locuri sunt libere pe segmentul lui și primește un bilet cu prețul calculat corect. Administratorul poate gestiona rutele, opririle, cursele si reducerile într-un singur loc, fără să repete aceleași informații, deci are mai puține șanse să greșească. Prin istoricul biletelor se poate vedea și cât de ocupate sunt cursele.

2. Obiective: funcționalități, operații, utilizatori

Utilizatori

Aplicația are două tipuri de utilizatori, cu nevoi diferite.

Clientul este persoana care vrea să călătorească cu trenul. El poate să caute curse între două stații, să vadă orarul unei curse și locurile libere dintr-o cursă, să cumpere un bilet și să îl anuleze dacă nu mai are nevoie de el. De asemenea, poate să își vadă istoricul biletelor cumpărate, adică toate călătoriile pe care le-a făcut sau pe care le are programate.

Administratorul este persoana care se ocupă de datele din sistem: introduce și actualizează stațiile, rutele cu opririle lor, trenurile cu vagoanele și locurile lor, cursele de pe fiecare zi, reducerile și coeficienții de preț ai tipurilor de tren.

Ambele tipuri de utilizatori se autentifică în aplicație cu email și parolă.

Funcționalități

Prima funcționalitate este căutarea curselor. Clientul alege stația de plecare, stația de sosire și data, iar aplicația îi arată doar cursele din ziua aleasă a căror rută oprește în ambele stații și la care stația de urcare este înaintea celei de coborâre.

A doua funcționalitate este afișarea orarului unei curse. Pentru fiecare cursă se pot vedea toate stațiile în care oprește ruta ei, în ordine, împreună cu orele de sosire și de plecare și cu distanța față de stația de origine. Rutele diferă între ele prin numărul de opriri: unele opresc în aproape toate stațiile, altele doar în orașele mari.

A treia funcționalitate este afișarea locurilor libere. Aplicația arată, pe vagoane, ce locuri mai sunt disponibile pentru cursa aleasă, ținând cont de stațiile între care va călători clientul. Un loc ocupat doar între două stații poate fi vândut altcuiva pentru restul traseului.

A patra funcționalitate este cumpărarea unui bilet. Clientul alege cursa și locul, iar prețul se calculează automat, în funcție de distanța parcursă, de tipul trenului, de clasa vagonului și de reducerea aplicată. Biletul reține clientul, cursa, locul, stațiile între care se călătorește, prețul plătit și data cumpărării.

A cincea funcționalitate este anularea unui bilet. Biletul nu se șterge, ci primește starea „anulat”, ca să rămână în istoric. Locul devine din nou liber pentru alți clienți.

A șasea funcționalitate este istoricul biletelor. Fiecare client poate vedea toate biletele pe care le-a cumpărat, cu cursa, locul, stațiile, prețul și starea lor.

Operații

În baza de date se pot adăuga, modifica și șterge stațiile, rutele, opririle, trenurile, vagoanele, locurile, cursele și reducerile. Se înregistrează clienții și administratorii, cu parola salvată doar sub formă de hash, iar la autentificare se verifică emailul și parola. Stergerea nu este permisa oriunde: statiile, cursele si locurile care apar deja in bilete nu pot fi sterse fara sa se strice istoricul, de aceea relatiile dintre tabele sunt protejate prin chei straine.

Pentru client, aplicația execută interogări de selecție. Căutarea curselor între două stații caută rutele care opresc în ambele stații, în ordinea corectă, și cursele lor din ziua aleasă. Orarul unei curse se obține din opririle rutei ei, în ordine, iar locurile libere se obțin scăzând din locurile trenului pe cele cu bilete valide care se suprapun cu segmentul ales. Istoricul unui client este lista biletelor lui, cu cursa, locul, stațiile, prețul și starea.

La cumpărarea unui bilet, baza de date verifică mai întâi dacă locul este liber: dacă mai există un bilet valid pe același loc, la aceeași cursă, cu segment care se suprapune, noul bilet este respins. Dacă locul este liber, se calculează prețul din distanța dintre stația de urcare și cea de coborâre, tipul trenului, clasa vagonului și reducerea aplicată, iar biletul se salvează într-o tranzacție. Astfel, același loc nu poate fi vândut de două ori, nici dacă doi clienți încearcă să îl cumpere în același timp. Când un bilet se anulează, starea lui devine „anulat”, iar locul apare din nou liber.

3. Proiectare

Analiza cerințelor

Acest capitol descrie ce trebuie să poată face aplicația și ce date trebuie să păstreze baza de date, înainte de proiectarea propriu-zisă. El pornește de la utilizatori și de la funcționalitățile lor și cu regulile din lumea reală pe care baza de date trebuie să le respecte și se încheie cu prioritizarea cerințelor.

Utilizatori

Aplicația are două tipuri de utilizatori, cu nevoi diferite. Clientul este persoana care vrea să călătorească cu trenul: caută curse, vede locurile libere, cumpără și anulează bilete și își consultă istoricul. Administratorul este persoana care se ocupă de datele din sistem: introduce și actualizează stațiile, rutele cu opririle lor, trenurile cu vagoanele și locurile lor, cursele, reducerile și coeficienții de preț ai tipurilor de tren.

Cerințe funcționale

FR1. Clientul poate căuta curse între două stații, într-o zi aleasă.

FR2. Aplicația afișează doar cursele din ziua respectivă a căror rută oprește în ambele stații, cu stația de urcare înaintea celei de coborâre.

FR3. Clientul poate vedea orarul unei curse, adică stațiile în care oprește ruta ei, în ordine, cu orele de sosire și de plecare.

FR4. Clientul poate vedea locurile libere pe vagoane pentru segmentul pe care vrea să-l parcurgă.

FR5. Clientul poate cumpăra un bilet pentru un loc anume.

FR6. Prețul biletului se calculează automat, în funcție de distanța parcursă, de tipul trenului, de clasa vagonului și de reducerea clientului, dacă are una.

FR7. Clientul poate anula un bilet; biletul rămâne în istoric cu starea „anulat”, iar locul devine din nou liber.

FR8. Clientul își poate vedea istoricul biletelor cumpărate.

FR9. Administratorul poate adăuga, modifica și șterge stațiile.

FR10. Administratorul poate adăuga, modifica și șterge rutele și opririle lor.

FR11. Administratorul poate adăuga, modifica și șterge trenurile, vagoanele și locurile.

FR12. Administratorul poate adăuga, modifica și șterge cursele.

FR13. Administratorul poate adăuga, modifica și șterge reducerile.

FR14. Utilizatorul (client sau administrator) se autentifică în aplicație cu adresa de email și cu parolă.

Cerințe nonfuncționale

NFR1. Integritatea datelor: informațiile trebuie să rămână corecte indiferent de operațiile efectuate, ceea ce se asigură prin chei primare și străine, constrângeri de unicitate și constrângeri de verificare.

NFR2. Consistența la cumpărări simultane: două cumpărări simultane pentru același loc, pe segmente care se suprapun, nu trebuie să ducă la vânzarea lui de două ori. Cumpărarea unui bilet se execută ca tranzacție, adică se salvează complet sau deloc.

NFR3. Păstrarea istoricului: biletele anulate nu se șterg, ci își schimbă starea, iar datele de care depind biletele (stații, curse, locuri) nu pot fi șterse cât timp sunt referite.

NFR4. Securitatea accesului: utilizatorii se autentifică cu email și parolă, iar parolele nu se păstrează în clar, ci doar sub formă de hash. Accesul la date este separat pe roluri, astfel încât clientul să-și vadă doar propriile bilete, iar administratorul să gestioneze datele de referință (stații, rute, trenuri, curse, reduceri).

Reguli de gestiune

Datele trebuie să respecte câteva reguli din lumea reală, pe care baza de date le impune prin constrângeri sau prin trigger-e.

RG1. Un loc nu poate fi vândut de două ori pe aceeași cursă pentru segmente care se suprapun, dar poate fi vândut din nou pentru un segment diferit.

RG2. Stația de urcare trebuie să fie înaintea celei de coborâre în traseul rutei, iar ambele stații trebuie să facă parte din ruta cursei.

RG3. Locul din bilet trebuie să aparțină unui vagon al trenului care face cursa.

RG4. Pe aceeași rută, o stație apare o singură dată, iar două opriri nu pot avea aceeași poziție în traseu.

RG5. Un bilet anulat nu se șterge, ci rămâne în istoric cu starea „anulat”, iar locul devine din nou disponibil.

RG6. Fiecare cursă este creată de un singur administrator, iar același tren nu poate face de două ori aceeași rută în aceeași zi.

RG7. Adresa de email este unică, atât pentru clienți, cât și pentru administratori, deoarece servește la autentificare.

RG8. Un bilet aparține unui singur client, este emis pentru o singură cursă, ocupă un singur loc și are cel mult o reducere.

RG9. Prețul unui bilet se calculează la cumpărare, din distanța dintre stația de urcare și cea de coborâre, tipul trenului, clasa vagonului și reducerea aplicată, și rămâne neschimbat chiar dacă reducerile sau coeficienții se modifică ulterior.

RG10. Întârzierea unei curse se înregistrează la nivelul cursei, prin stare și minute de întârziere, și nu modifică orarul rutei.

Regulile RG1-RG3 se implementează prin trigger sau procedură stocată, pentru că presupun compararea mai multor rânduri sau tabele. Celelalte se implementează prin constrângeri: chei primare și străine, UNIQUE și CHECK.

Prioritizarea cerințelor: Metoda MoSCoW

Cerințele au fost prioritizate cu metoda MoSCoW, care le împarte în patru categorii: obligatorii, importante, opționale și nerealizate în această versiune.

Sunt obligatorii cerințele fără de care aplicația nu poate funcționa conform scopului: autentificarea în aplicație cu email și parolă, păstrarea stațiilor, a rutelor cu opriri, a trenurilor cu vagoane și locuri, a curselor, a clienților și a biletelor. De asemenea, sunt obligatorii introducerea și actualizarea datelor de către administrator, căutarea curselor între două stații, afișarea locurilor libere pe segmentul ales, cumpărarea unui bilet, regula ca același loc nu se vinde de două ori pentru segmente suprapuse, precum și calculul automat al prețului (în funcție de distanță, tip de tren și clasă) și aplicarea reducerilor.

Sunt importante funcționalitățile: anularea biletelor, istoricul biletelor și afișarea orarului unei curse. De asemenea, sunt importante înregistrarea stării și a întârzierilor curselor și integrarea bazei de date într-un flux de inteligență artificială.

Sunt opționale afișarea rutei în timp real într-o aplicație mobilă și trimiterea de notificări push automate către client atunci când trenul înregistrează o întârziere.

Nu vor fi realizate în această versiune: plata reală a biletelor, care este doar simulată, biletele de grup și abonamentele, călătoriile cu schimbare de tren (legăturile dintre bilete) și realocarea automată a clienților, gestiunea personalului și a locomotivelor, preluarea automată a întârzierilor prin sisteme GPS live.

Descrierea entităților

Modelul conceptual include 11 entități fundamentale, proiectate pentru a acoperi complet vânzarea biletelor de tren, de la traseele și trenurile disponibile până la clienții care cumpără bilete:

STATII: locurile în care pot opri trenurile.
Atribute:

- id_statie: identificatorul unic al stației;
- nume: denumirea stației;
- oras: orașul în care se află stația;
- judet: județul în care se află stația.

RUTE: traseele pe care circulă trenurile, de exemplu Baia Mare - Cluj. O rută descrie doar traseul, adică șirul de stații parcurse în ordine, și nu are dată sau tren, pentru că traseul și orele sunt aceleași în fiecare zi.
Atribute:

- id_ruta: identificatorul unic al rutei;
- denumire: numele rutei.

TIPURI_TREN: categoriile de tren (InterCity, InterRegio, Regio). Un tip grupează mai multe trenuri.
Atribute:

- id_tip: identificatorul unic al tipului de tren;
- denumire: numele categoriei;
- coeficient_pret: coeficientul care influențează prețul biletelor.

TRENURI: trenurile ca vehicule. Un tren aparține unui singur tip și are mai multe vagoane. El nu are un traseu propriu, pentru că traseul vine de la rută, prin intermediul cursei.
Atribute:

- id_tren: identificatorul unic al trenului;
- nume: numele sau numărul trenului.

VAGOANE: vagoanele unui tren. Un vagon conține mai multe locuri.
Atribute:

- id_vagon: identificatorul unic al vagonului;
- nr_vagon: numărul vagonului în cadrul trenului;
- clasa: clasa vagonului (1 sau 2), care influențează prețul.

LOCURI: locurile dintr-un vagon. Biletul se cumpără pentru un loc anume.
Atribute:

- id_loc: identificatorul unic al locului;
- nr_loc: numărul locului în cadrul vagonului.

CURSE: o rută parcursă într-o anumită zi de un anumit tren. Informațiile care depind de zi se păstrează aici, nu la rută. Fiecare cursă este creată de un singur administrator.
Atribute:

- id_cursa: identificatorul unic al cursei;
- data_cursa: ziua în care se desfășoară cursa;
- status: starea cursei (la timp, întârziată, anulată);
- intarziere_minute: numărul de minute de întârziere.

ADMINISTRATORI: persoanele care introduc și actualizează datele din sistem. Un administrator poate crea mai multe curse.
Atribute:

- id_admin: identificatorul unic al administratorului;
- nume: numele de familie al administratorului;
- prenume: prenumele administratorului;
- parola_hash: parola contului, salvata doar sub forma de hash;
- email: adresa de email a administratorului.

CLIENȚI: persoanele care cumpără bilete. Un client poate cumpăra mai multe bilete.
Atribute:

- id_client: identificatorul unic al clientului;
- nume: numele de familie al clientului;
- prenume: prenumele clientului
- email: adresa de email a clientului, unică;
- parola_hash: parola contului, salvata doar sub forma de hash;
- nr_telefon: număr de contact.

REDUCERI: categoriile de călători care primesc un preț redus, precum elevii, studenții sau pensionarii. O reducere se poate aplica mai multor bilete.
Atribute:

- id_reducere: identificatorul unic al reducerii;
- denumire: numele categoriei de călători;
- procent: procentul cu care se reduce prețul.

BILETE: entitatea centrală a modelului. Un bilet este cumpărat de un client, este pentru o cursă și ocupă un loc, iar stația de urcare și cea de coborâre stabilesc segmentul parcurs. El

poate avea aplicată o reducere.
Atribute:

- id_bilet: identificatorul unic al biletului;
- pret_platit: prețul final plătit la cumpărare;
- status_bilet: starea biletului (valid sau anulat);
- data_cumpararii: momentul în care a fost cumpărat biletul.

Între entitățile RUTE și STAȚII există o relație de tip N-M, numită „oprește_în“: o rută oprește în mai multe stații, iar o stație face parte din mai multe rute. Relația are atribute proprii:

- ordine: poziția opririi în traseul rutei;
- ora_sosire: ora la care ajunge trenul în stație;
- ora_plecare: ora la care pleacă trenul din stație;
- distanta: distanța față de stația de origine, în kilometri.
  La proiectarea modelului logic ea va deveni tabelul Opriri.

Descrierea relațiilor

Modelul conceptual conține treisprezece relații. Pentru fiecare este precizat între ce entități există, ce cardinalitate are, ce înseamnă și cum va fi implementată în modelul logic.

Relația se clasifică leagă entitățile Tipuri de tren și Trenuri și este de tip 1-N. Un tip de tren (InterCity, InterRegio, Regio) grupează mai multe trenuri, iar fiecare tren aparține unui singur tip. Prin ea, regulile comune unui tip, cum este coeficientul de preț, se păstrează într-un singur loc. În modelul logic, tabelul Trenuri primește cheia străină id_tip.

Relația are leagă entitățile Trenuri și Vagoane și este de tip 1-N. Un tren are mai multe vagoane, iar un vagon aparține unui singur tren. În modelul logic, tabelul Vagoane primește cheia străină id_tren.

Relația conține leagă entitățile Vagoane și Locuri și este de tip 1-N. Un vagon conține mai multe locuri, iar un loc se află într-un singur vagon. În modelul logic, tabelul Locuri primește cheia străină id_vagon.

Relația oprește în leagă entitățile Rute și Stații și este de tip N-M. O rută oprește în mai multe stații, iar o stație face parte din mai multe rute. Relația are atribute proprii, anume ordinea opririi în traseu, ora de sosire, ora de plecare și distanța față de stația de origine, deoarece aceste informații depind de perechea rută-stație, nu de una singură dintre ele. Fiind o relație N-M, ea nu poate fi implementată direct și devine în modelul logic tabelul Opriri, care conține cheile străine id_ruta și id_statie.

Relația parcurge leagă entitățile Rute și Curse și este de tip 1-N. O rută este parcursă de mai multe curse, în zile diferite, iar fiecare cursă se desfășoară pe o singură rută. În modelul logic, tabelul Curse primește cheia străină id_ruta.

Relația circulă leagă entitățile Trenuri și Curse și este de tip 1-N. Un tren efectuează mai multe curse, iar fiecare cursă este efectuată de un singur tren. Împreună cu relația parcurge, ea face ca o cursă să fie o rută, într-o zi anume, cu un anumit tren. În modelul logic, tabelul Curse primește cheia străină id_tren.

Relația creează leagă entitățile Administratori și Curse și este de tip 1-N. Un administrator creează mai multe curse, iar fiecare cursă este creată de un singur administrator. În modelul logic, tabelul Curse primește cheia străină id_admin.

Relația pentru leagă entitățile Curse și Bilete și este de tip 1-N. Pentru o cursă se vând mai multe bilete, iar fiecare bilet este emis pentru o singură cursă. În modelul logic, tabelul Bilete primește cheia străină id_cursa.

Relația ocupă leagă entitățile Locuri și Bilete și este de tip 1-N. Un loc poate apărea în mai multe bilete, fie pentru curse diferite, fie pentru segmente care nu se suprapun pe aceeași cursă, dar fiecare bilet ocupă un singur loc. În modelul logic, tabelul Bilete primește cheia străină id_loc.

Relațiile urcare și coborâre leagă fiecare entitățile Stații și Bilete și sunt de tip 1-N. O stație poate fi stație de urcare pentru mai multe bilete și stație de coborâre pentru altele, dar fiecare bilet are exact o stație de urcare și exact una de coborâre. Împreună ele definesc segmentul parcurs de călător, pe baza căruia se verifică dacă două bilete se suprapun pe același loc. Entitățile sunt legate de două ori, deci în modelul logic tabelul Bilete primește două chei străine către Stații: id_statie_urcare și id_statie_coborare.

Relația cumpără leagă entitățile Clienți și Bilete și este de tip 1-N. Un client poate cumpăra mai multe bilete, iar fiecare bilet este cumpărat de un singur client. Prin ea se obține istoricul biletelor unui client. În modelul logic, tabelul Bilete primește cheia străină id_client.

Relația se aplică leagă entitățile Reduceri și Bilete și este de tip 1-N. O reducere (de exemplu pentru elevi, studenți sau pensionari) se aplică la mai multe bilete, iar un bilet are aplicată cel mult o reducere. În modelul logic, tabelul Bilete primește cheia străină id_reducere.

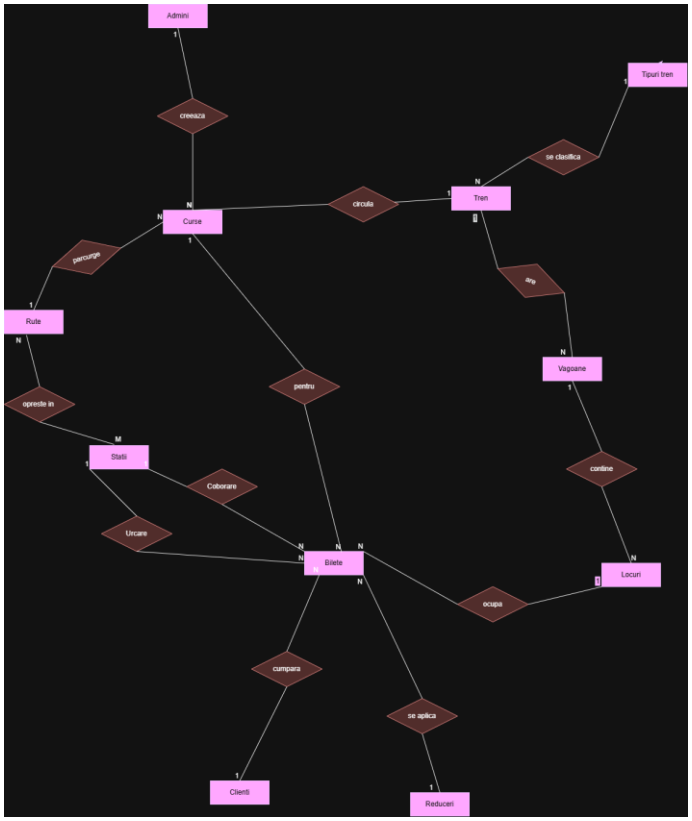


Figura 3.1 Diagrama ER a aplicației

Modelul Logic

Modelul logic este varianta modelului conceptual scrisă în tabele. Fiecare entitate a devenit un tabel, iar atributele ei au devenit coloane. Relațiile 1-N au fost transformate în chei străine puse în tabelul din partea N, iar relația N-M dintre rute și stații a devenit tabelul Opriri. Modelul are 12 tabele și este independent de SGBD.

Dicționarul de date

Dicționarul descrie fiecare tabel al modelului logic: coloanele, tipul de date, constrângerile și semnificația lor.

Abrevieri: PK = cheie primara, FK = cheie straina, UQ = unic (UNIQUE), NN = NOT NULL.

Tabelul 3.1 Dicționar de date -Admini

Administratorii care introduc si actualizeaza datele din sistem.

Coloana	Tip	Constrangeri	Descriere
id_admin	INT	PK	Identificator unic al administratorului
nume	VARCHAR(50)	NN	Numele de familie
prenume	VARCHAR(50)	NN	Prenumele
email	VARCHAR(100)	NN, UQ	Adresa de email
parola_hash	VARCHAR(255)	NN	Parola, salvata doar ca hash

Tabelul 3.2 Dicționar de date -Rute

Traseele pe care circulă trenurile, fără dată sau tren.

Coloana	Tip	Constrangeri	Descriere
id_ruta	INT	PK	Identificator unic al rutei
denumire	VARCHAR(100)	NN, UQ	Numele rutei, de exemplu Baia Mare - Cluj

Tabelul 3.3 Dicționar de date -Statii

Locurile in care pot opri trenurile.

Coloana	Tip	Constrangeri	Descriere
id_statie	INT	PK	Identificator unic al statiei
nume	VARCHAR(100)	NN	Denumirea statiei
oras	VARCHAR(100)	NN	Orasul in care se afla statia
judet	VARCHAR(50)	NN	Judetul

Constrangere de tabel: UQ (nume, oras).

Tabelul 3.4 Dicționar de date -Opriri

Opririle fiecarei rute, adica tabelul de legatura dintre Rute si Statii.

Coloana	Tip	Constrangeri	Descriere
id_oprire	INT	PK	Identificator unic al opririi
id_ruta	INT	FK → Rute, NN	Ruta careia ii apartine oprirea
id_statie	INT	FK → Statii, NN	Statia in care se opreste
ordine	SMALLINT	NN, CHECK (ordine >= 1)	Pozitia opririi in traseu
ora_sosire	TIME	poate fi NULL	Ora sosirii (lipseste la prima oprire)
ora_plecare	TIME	poate fi NULL	Ora plecarii (lipseste la ultima oprire)
distanta	DECIMAL(6,1)	NN, CHECK (distanta >= 0)	Kilometri fata de statia de origine a rutei

Constrangeri de tabel: UQ (id_ruta, id_statie) si UQ (id_ruta, ordine).

Tabelul 3.5 Dicționar de date –Trenuri

Trenurile ca vehicule.

Coloana	Tip	Constrangeri	Descriere
id_tren	INT	PK	Identificator unic al trenului
id_tip	INT	FK → Tipuri_tren, NN	Tipul trenului
nume	VARCHAR(50)	NN, UQ	Numele sau numarul trenului

Tabelul 3.6 Dicționar de date -Tipuri_tren

Categoriile de tren (InterCity, InterRegio, Regio).

Coloana	Tip	Constrangeri	Descriere
id_tip	INT	PK	Identificator unic al tipului
denumire	VARCHAR(30)	NN, UQ	Numele categoriei
coeficient_pret	DECIMAL(4,2)	NN, CHECK (coeficient_pret > 0)	Coeficient folosit la calculul pretului

Tabelul 3.7 Dicționar de date -Vagoane

Vagoanele unui tren.

Coloana	Tip	Constrangeri	Descriere
id_vagon	INT	PK	Identificator unic al vagonului
id_tren	INT	FK → Trenuri, NN	Trenul din care face parte
nr_vagon	SMALLINT	NN, CHECK (nr_vagon >= 1)	Numarul vagonului in tren
clasa	SMALLINT	NN, CHECK (clasa IN (1, 2))	Clasa vagonului

Constrangere de tabel: UQ (id_tren, nr_vagon).

Tabelul 3.8 Dicționar de date - Locuri

Locurile dintr-un vagon.

Coloana	Tip	Constrangeri	Descriere
id_loc	INT	PK	Identificator unic al locului
id_vagon	INT	FK → Vagoane, NN	Vagonul in care se afla
nr_loc	SMALLINT	NN, CHECK (nr_loc >= 1)	Numarul locului in vagon

Constrangere de tabel: UQ (id_vagon, nr_loc).

Tabelul 3.9 Dicționar de date - Curse

O ruta parcursa intr-o zi anume de un tren anume.

Coloana	Tip	Constrangeri	Descriere
id_cursa	INT	PK	Identificator unic al cursei
id_admin	INT	FK → Admini, NN	Administratorul care a creat cursa

id_ruta	INT	FK → Rute, NN	Ruta parcursa
id_tren	INT	FK → Trenuri, NN	Trenul care face cursa
data_cursa	DATE	NN	Ziua in care se desfasoara cursa
status	VARCHAR(20)	NN, DEFAULT 'la timp', CHECK (status IN ('la timp', 'intarziata', 'anulata'))	Starea cursei
intarziere_minute	SMALLINT	NN, DEFAULT 0, CHECK (intarziere_minute >= 0)	Minutele de intarziere

Constrangere de tabel: UQ (id_ruta, id_tren, data_cursa).

Tabelul 3.10 Dicționar de date - Clienti

Persoanele care cumpara bilete.

Coloana	Tip	Constrangeri	Descriere
id_client	INT	PK	Identificator unic al clientului
nume	VARCHAR(50)	NN	Numele de familie
prenume	VARCHAR(50)	NN	Prenumele
parola_hash	VARCHAR(255)	NN	Parola, salvata doar ca hash
email	VARCHAR(100)	NN, UQ	Adresa de email
nr_telefon	VARCHAR(15)	poate fi NULL	Numar de contact

Tabelul 3.11 Dicționar de date - Reduceri

Categoriile de calatori cu pret redus.

Coloana	Tip	Constrangeri	Descriere
id_reducere	INT	PK	Identificator unic al reducerii
denumire	VARCHAR(50)	NN, UQ	Numele categoriei (elev, student, pensionar)
procent	DECIMAL(5,2)	NN, CHECK (procent BETWEEN 0 AND 100)	Procentul cu care scade pretul

Tabelul 3.12 Dicționar de date - Bilete

Biletele cumparate de clienti, entitatea centrala.

Coloana	Tip	Constrangeri	Descriere
id_bilet	INT	PK	Identificator unic al biletului
id_client	INT	FK → Clienti, NN	Clientul care a cumparat biletul
id_cursa	INT	FK → Curse, NN	Cursa pentru care e emis biletul
id_loc	INT	FK → Locuri, NN	Locul ocupat
id_statie_urcare	INT	FK → Statii, NN	Statia de urcare
id_statie_coborare	INT	FK → Statii, NN	Statia de coborare
id_reducere	INT	FK → Reduceri, poate fi NULL	Reducerea aplicata (NULL = fara reducere)
pret_platit	DECIMAL(8,2)	NN, CHECK (pret_platit >= 0)	Pretul final, fixat la cumparare
status_bilet	VARCHAR(10)	NN, DEFAULT 'valid', CHECK (status_bilet IN ('valid', 'anulat'))	Starea biletului
data_cumpararii	TIMESTAMP	NN, DEFAULT momentul curent	Momentul cumpararii

Constrangere de tabel: CHECK (id_statie_urcare <> id_statie_coborare).

Normalizarea bazei de date

Scopul procesului de normalizare este de a structura datele astfel încât să se elimine redundanța și să se prevină anomaliile care pot apărea la operațiile de inserare, actualizare sau ștergere. Schema bazei de date a acestei aplicații a fost proiectată respectând primele trei forme normale (FN1, FN2 și FN3).

1. Prima formă normală
O bază de date este în FN1 dacă toate valorile din coloane sunt atomice și fiecare tabel are o cheie primară unică.

În baza de date a aplicației, informațiile care ar putea fi păstrate ca liste sunt salvate pe rânduri separate. Opririle unei rute nu stau într-o singură coloană cu toate stațiile, ci în tabelul Opriri, unde fiecare rând este o oprire. La fel, locurile unui vagon sunt în tabelul Locuri, iar biletele unui client sunt în tabelul Bilete, câte un rând pentru fiecare bilet. Numele și prenumele sunt salvate în coloane separate, atât în Admini, cât și în Clienti, iar fiecare coloană conține o singură valoare. Toate cele 12 tabele au o cheie primară formată dintr-o singură coloană (id_admin, id_ruta, id_statie, id_oprire, id_tip, id_tren, id_vagon, id_loc, id_cursa, id_client, id_reducere, id_bilet).

2. A doua formă normală
O bază de date este în FN2 dacă este în FN1 și toate coloanele care nu sunt chei depind de întreaga cheie primară.

Tabelele cu cheie primară dintr-o singură coloană respectă automat FN2, deoarece nu pot exista dependențe de o parte a cheii. În tabelul Opriri, atributele ordine, ora_sosire, ora_plecare și distanta depind de perechea formată din rută și stație: aceeași stație are o altă poziție și alte ore pe o altă rută, iar aceeași rută are ore diferite în stații diferite. De aceea ele sunt păstrate în Opriri, datele generale despre rută rămân în Rute, iar datele despre stație rămân în Statii. În Vagoane, clasa depinde de perechea tren și număr de vagon, pentru că același număr de vagon poate avea altă clasă la un alt tren.

3. A treia formă normală
O bază de date este în FN3 dacă este în FN2 și niciun atribut care nu este cheie nu depinde de alt atribut care nu este cheie.

În Trenuri reținem doar id_tip, nu și denumirea sau coeficientul tipului de tren. Aceste date sunt salvate o singură dată în Tipuri_tren, deci dacă se schimbă un coeficient de preț, îl modificăm într-un singur loc. În Curse reținem doar id_ruta, id_tren și id_admin, fără să duplicăm denumirea rutei, numele trenului sau datele administratorului. În Bilete reținem doar cheile id_client, id_cursa, id_loc, id_statie_urcare, id_statie_coborare și id_reducere, fără să repetăm numele clientului, numele stațiilor sau procentul reducerii. În Admini și Clienti, emailul și parola (salvată ca hash) depind direct de cheia primară, iar emailul este unic. Prețul plătit (pret_platit) se păstrează în Bilete pentru că este o valoare fixată la momentul cumpărării, care trebuie să rămână neschimbată chiar dacă reducerile sau coeficienții se modifică ulterior, deci nu este o informație duplicată.

Concluzie: Baza de date este complet normalizată până la forma normală 3.

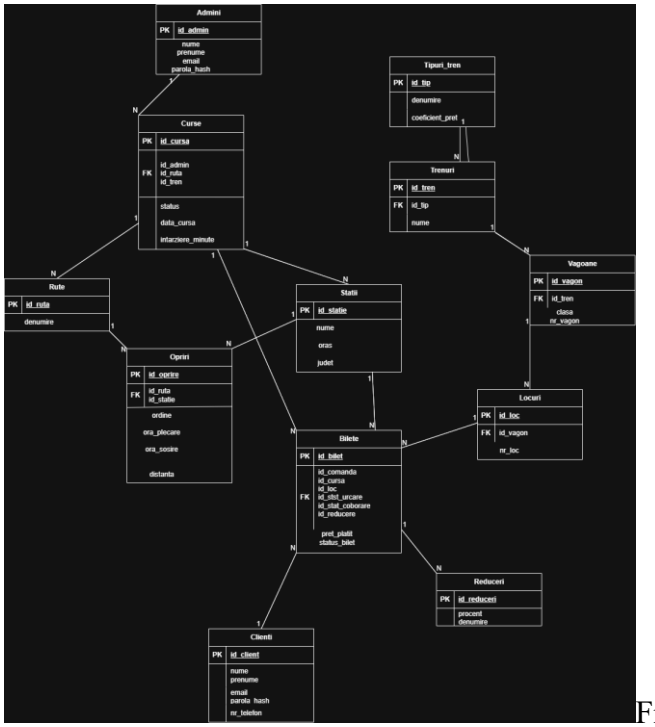


Figura 3.2 Modelul logic al aplicației